

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Práctica:

Diseño y toma de decisiones

PONDERACIÓN: 4 pts

 **Tiempo estimado: 10 hrs/min**

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART	3
3. Enunciado de la Práctica	4
3.1 Descripción del problema a resolver	4
3.2 Alcance de la práctica	4
4. Entregables	5
5. Material de apoyo	5
6. Recursos y herramientas a utilizar	5
7. Cronograma	6
8. Rúbrica de Calificación	6
8.1 Requisitos para optar a la calificación	6
8.2 Resumen de Puntuaciones	7
Detalle de la Calificación	8
Comentarios Generales	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad Profesional y Ética en el Desarrollo de Software	El estudiante deberá diseñar e implementar una arquitectura basada en microservicios cumpliendo estándares de calidad, buenas prácticas de desarrollo, uso correcto de contenedores Docker y documentación técnica clara. Además, deberá respetar las normas de originalidad y trabajo individual.

1.2. Competencia(s)

Tipo de competencia	Arquitectónica y de Diseño de Software Empresarial
Competencia General	Aplicar arquitecturas modernas de desarrollo de software para construir sistemas escalables, mantenibles y desacoplados utilizando tecnologías actuales.
Competencia Específica	Diseñar e implementar una arquitectura basada en microservicios utilizando Docker, múltiples lenguajes de programación, integrados mediante un API Gateway.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- Individual: Autonomía técnica en la toma de decisiones de diseño, al elegir libremente la temática del sistema y los lenguajes de programación a utilizar en cada microservicio.
- Individual: Responsabilidad y gestión del tiempo, al coordinar la implementación de múltiples servicios independientes dentro de un plazo definido.
- Individual: Comunicación técnica escrita, mediante la elaboración de documentación completa que incluye diagramas de arquitectura, ER, despliegue y contratos de microservicios.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Diseñar e implementar un	Implementar mínimo 4	Utilizando herramientas	Fortalecer competencias	Antes del x de Julio de 2026 a

sistema de temática libre basado en al menos 4 microservicios funcionales, contenedorizados con Docker e integrados mediante un API Gateway, incorporando el servicio de autenticación de la Práctica 2.	microservicios con al menos 1 endpoint funcional cada uno, usando 2 lenguajes de programación distintos, todos funcionando mediante Docker Compose, con GraphQL implementado en al menos 2 servicios.	como Docker, Docker Compose, lenguajes modernos (Node.js, Java, Python, Go, etc.) y repositorios en la nube.	en arquitectura distribuida y contenedores, preparando al estudiante para entornos empresariales modernos.	las 23:59 hrs, entregando el repositorio vía UEDI.
--	---	--	--	--

3. Enunciado de la Práctica.

3.1 Descripción del problema a resolver

El estudiante deberá diseñar e implementar un sistema basado en **arquitectura de microservicios con temática libre**. El objetivo es simular un entorno empresarial moderno basado en contenedores y arquitectura distribuida.

El sistema deberá cumplir con los siguientes requerimientos:

- Estar compuesto por **mínimo 4 microservicios** funcionales.
- **Integrar el servicio de autenticación** desarrollado en la Práctica 2.
- Cada microservicio debe tener **al menos un endpoint funcional**.
- **Implementar Docker** para cada servicio mediante un Dockerfile individual.
- Implementar un **API Gateway funcional** que consuma todos los microservicios.
- Utilizar **al menos 2 lenguajes de programación distintos** entre los microservicios.
- Implementar **Docker Compose** para levantar todo el sistema con un solo comando.
- Implementar **GraphQL en al menos 2 servicios**.

3.2 Alcance de la práctica

Obligatorio:

- 4 microservicios mínimo, cada uno con al menos 1 endpoint funcional.
- Dockerfile individual por cada microservicio.
- Archivo `docker-compose.yml` funcional que levante todo el sistema.
- API Gateway funcional que enrute hacia todos los microservicios.
- Uso de al menos 2 lenguajes de programación distintos.
- GraphQL implementado en al menos 2 servicios.
- Diagrama de arquitectura (comunicación entre servicios).

- Diagrama de despliegue.
- Diagrama ER completo de las bases de datos.
- Documentación técnica completa.
- Repositorio en la nube con la estructura solicitada.
- Contrato de microservicios en Swagger o Postman.

No requerido (pero recomendado):

- Comunicación directa entre microservicios (sin pasar por el Gateway).
- Implementación de base de datos separada por servicio.

3.3 Requerimientos técnicos

- **Contenedores: Docker y Docker Compose (obligatorio).**
- **Repositorio: En la nube (GitHub, GitLab, Bitbucket) con nombre **Practicas-SA-<<SECCIÓN>>-<<CARNE>>** y carpeta **P4**.**
- **Lenguajes: Mínimo 2 distintos entre los microservicios (Node.js, Java, Python, Go u otros).**
- **API: GraphQL implementado en al menos 2 servicios.**
- **Diseño: Principios de desacoplamiento entre microservicios.**
- **Contrato: Definición de endpoints documentada en Swagger o Postman.**

4. Entregables

Tipo	Descripción
Repositorio en la nube	Repositorio privado con el código fuente de la API REST, organizado dentro de la carpeta P4 .
README en Markdown	Documento que incluye: explicación con palabras propias de los 5 principios SOLID y evidencia de su aplicación en el código (archivo/clase, justificación y fragmentos de código reales).
Carpeta P4	Código fuente completo de todos los microservicios con su respectiva documentación.
Dockerfile	Un Dockerfile funcional por cada microservicio.
docker-compose.yml	Archivo funcional que levante todo el sistema con un solo comando.
Enlace en UEDI	Subir el enlace del repositorio a la plataforma UEDI antes de la fecha límite.

5. Material de apoyo

- [Documentación oficial de Docker](#)
- [Documentación de Docker Compose](#)
- Martin Fowler — *Microservices Architecture* (martinfowler.com)
- [Documentación oficial de GraphQL](#)
- [Guía de API Gateway Pattern — Microsoft](#)
- [Documentación de Swagger / OpenAPI](#)

6. Recursos y herramientas a utilizar

- **Software:** Docker Desktop, Docker Compose.
- **Lenguajes:** Node.js / Java / Python / Go (mínimo 2 distintos).
- **API:** GraphQL.
- **Plataformas:** GitHub / GitLab (repositorio), UEDI (entrega).
- **Documentación de contratos:** Swagger / Postman.
- **Lecturas recomendadas:** Documentación oficial de Docker, Martin Fowler – *Microservices Architecture*.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	XX/07/2026	XX/07/2026
Elaboración	XX/07/2026	XX/07/2026
Calificación	XX/07/2026	XX/07/2026

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
------	-------------	----------------

Uso de Docker	Todos los servicios están dockerizados con su propio Dockerfile.	
API Gateway	El Gateway consume y enruta correctamente a todos los microservicios.	
GraphQL	Implementado en al menos 2 microservicios.	
Repositorio	Cumple con la estructura y formato de nombre solicitados.	
Documentación	Completa, organizada e incluye diagramas y contratos de microservicios.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Documentación técnica completa	10	
Diagrama de Arquitectura, Despliegue y ER	10	
Organización del repositorio	5	
Preguntas teóricas	15	
Sub-Total Habilidades	40	
2. Conocimiento		
Implementación de 4 microservicios funcionales	20	
Dockerización correcta (Dockerfile por servicio)	10	
API Gateway funcional	10	

Docker Compose funcional	10	
Uso de 2 lenguajes distintos	10	
Sub-Total Conocimiento	60	
TOTAL	100	

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Calidad del diseño arquitectónico	10	Descripción	La documentación está completa, organizada e incluye los diagramas de arquitectura, despliegue, ER y contratos de microservicios en Swagger o Postman	La documentación está incompleta, desordenada o le faltan elementos requeridos (diagramas, contratos).	
			Rango del punteo	20-13 pts	12-0 pts	
1.2	Modelo ER correcto (por microservicio)	10		Los tres diagramas son coherentes, completos y reflejan fielmente la implementación entregada.	Faltan diagramas, son incorrectos o no corresponden con el sistema implementado .	
1.3	Separación adecuada de microservicios	5		El repositorio cumple el formato de nombre solicitado, la carpeta P4 contiene código y documentación bien estructurados.	El repositorio no cumple el formato o la organización del código y documentación es deficiente.	
1.4	Preguntas teóricas	15		El estudiante responde de manera puntual y concisa las preguntas teóricas sobre	El estudiante no responde correctamente las preguntas o	

				microservicios, Docker y GraphQL, demostrando comprensión del sistema implementado.	sus respuestas demuestran desconocimiento del sistema entregado.	
	Sub-Total de Puntos	40				
2	Conocimientos	60				
2.1	Implementación de 4 microservicios funcionales con al menos 1 endpoint cada uno, integración de autenticación P2 y GraphQL en al menos 2 servicios	20	Descripción	Los 4 microservicios están implementados, funcionales, cada uno responde correctamente a sus endpoints, GraphQL está implementado en al menos 2 servicios y el módulo de autenticación de P2 está integrado.	Algún microservicio no funciona, los endpoints fallan, GraphQL no está implementado o la autenticación de P2 no está integrada.	
			Rango del punteo	10 - 7 pts	6 - 0 pts	
2.2	Dockerización correcta (Dockerfile funcional por cada microservicio)	10		Cada microservicio tiene su propio Dockerfile correctamente configurado, la imagen se construye sin errores y el servicio levanta correctamente.	Algún Dockerfile falta, genera errores al construir o el servicio no levanta correctamente.	
2.3	API Gateway funcional	10		El API Gateway enruta correctamente las peticiones hacia todos los microservicios y responde con los datos esperados.	El API Gateway no enruta correctamente, falla al consumir algún microservicio o no está implementado.	
2.4	Docker Compose funcional	10		El archivo <code>docker-compose.yml</code> levanta todo el sistema correctamente con	El archivo <code>docker-compose.yml</code> falla al	

				un solo comando, sin errores de configuración.	ejecutarse, no levanta todos los servicios o presenta errores de configuración.	
2.5	Uso de al menos 2 lenguajes de programación distintos	10		Al menos 2 microservicios están implementados en lenguajes de programación diferentes, evidenciando correctamente el uso de cada uno.	Solo se usa un lenguaje de programación en todos los microservicios.	
	Sub-Total de Puntos	60				
	Total	100				